

- **Name:** Aj Kean Edoria
- **Term:** 2nd Term S.Y 25-26

Machine Problem 006: Time Response of Second-Order Systems

The [Python Control Systems Library](#) provides tools to simulate and analyze the time response of control systems. In this notebook, we use it to study the **step response** of second-order systems — including how pole locations determine the form of the response, how to compute the **natural frequency** and **damping ratio**, and how to measure key underdamped response specifications.

Consult the [Python Control Systems Documentation](#) for more details.

Installation

The [Python Control Systems Library](#) is not a standard part of most Python distributions. Run the following commands once to install the required packages.

To install both the Control Systems library and Slycot in an existing conda environment, run:

```
!conda install -c conda-forge control slycot
```

```
In [ ]: !conda install -c conda-forge slycot
!pip install control
!pip install seaborn
```

Library Usage

The control systems library is designed to work with a simplified syntax where libraries are imported without the standard prefixes.

```
In [1]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import control.matlab as control
```

Demonstration

General Second-Order Transfer Function

A **second-order system** has a transfer function of the form:

$$G(s) = \frac{b}{s^2 + as + b}$$

Unlike a first-order system where the parameter a only affects the speed of response, **changing the parameters a and b in a second-order system can change the entire form of the response.**

The behavior of the system depends on the **poles** — the roots of the characteristic equation:

$$s^2 + as + b = 0 \implies s = \frac{-a \pm \sqrt{a^2 - 4b}}{2}$$

The **discriminant** $\Delta = a^2 - 4b$ determines the nature of the poles and thus the form of the step response:

Condition	Poles	Response Type
$a = 0$	Purely imaginary: $\pm j\omega$	Undamped (oscillatory)
$\Delta > 0$	Two real, distinct	Overdamped
$\Delta = 0$	Two real, equal	Critically damped
$\Delta < 0$	Complex conjugate	Underdamped (decaying oscillation)

The Four Types of Step Responses

We now demonstrate all four response types by varying the parameter a while keeping $b = 9$.

Undamped: $a = 0 \rightarrow G(s) = \frac{9}{s^2 + 9}$

Poles: $s = \pm j3$ (purely imaginary)

Overdamped: $a = 9 \rightarrow G(s) = \frac{9}{s^2 + 9s + 9}$

Discriminant: $81 - 36 = 45 > 0 \rightarrow$ two real, distinct poles

Critically Damped: $a = 6 \rightarrow G(s) = \frac{9}{s^2 + 6s + 9}$

Discriminant: $36 - 36 = 0 \rightarrow$ two real, equal poles

Underdamped: $a = 2 \rightarrow G(s) = \frac{9}{s^2 + 2s + 9}$

Discriminant: $4 - 36 = -32 < 0 \rightarrow$ complex conjugate poles

```
In [2]: b = 9 # fixed numerator / b coefficient

systems = {
    'Undamped (a=0)': control.tf([b], [1, 0, b]),
    'Underdamped (a=2)': control.tf([b], [1, 2, b]),
    'Critically Damped (a=6)': control.tf([b], [1, 6, b]),
    'Overdamped (a=9)': control.tf([b], [1, 9, b]),
}
```

```

colors = ['blue', 'green', 'orange', 'red']

plt.figure(figsize=(9, 5))
for (label, G), color in zip(systems.items(), colors):
    poles = control.pole(G)
    print(f'{label}: poles = {np.round(poles, 4)}')
    y, t = control.step(G)
    plt.plot(t, y, color=color, linewidth=2, label=label)

plt.axhline(1, color='gray', linestyle=':', linewidth=1)
plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title('Step Responses of Second-Order Systems (b = 9)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

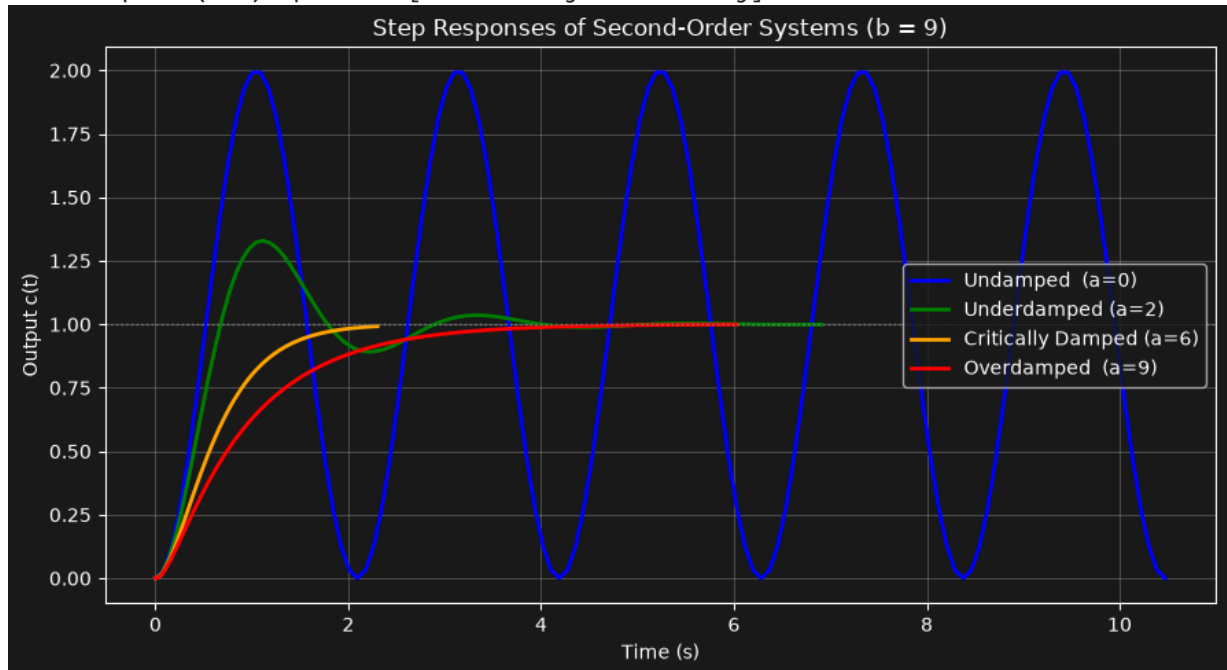
```

Undamped ($a=0$): poles = $[-0.+3.j \ 0.-3.j]$

Underdamped ($a=2$): poles = $[-1.+2.8284j \ -1.-2.8284j]$

Critically Damped ($a=6$): poles = $[-3.+0.j \ -3.-0.j]$

Overdamped ($a=9$): poles = $[-7.8541+0.j \ -1.1459+0.j]$



We can also visualize the pole locations for each system on the s -plane. Note how the position of the poles relative to the imaginary axis governs the response type.

```

In [3]: fig, ax = plt.subplots(figsize=(7, 5))
        markers = ['o', 's', '^', 'D']

        for (label, G), color, marker in zip(systems.items(), colors, markers):
            poles = control.pole(G)
            ax.plot(poles.real, poles.imag, marker, markersize=10,
                    markerfacecolor='none', markeredgewidth=2,
                    color=color, label=label)

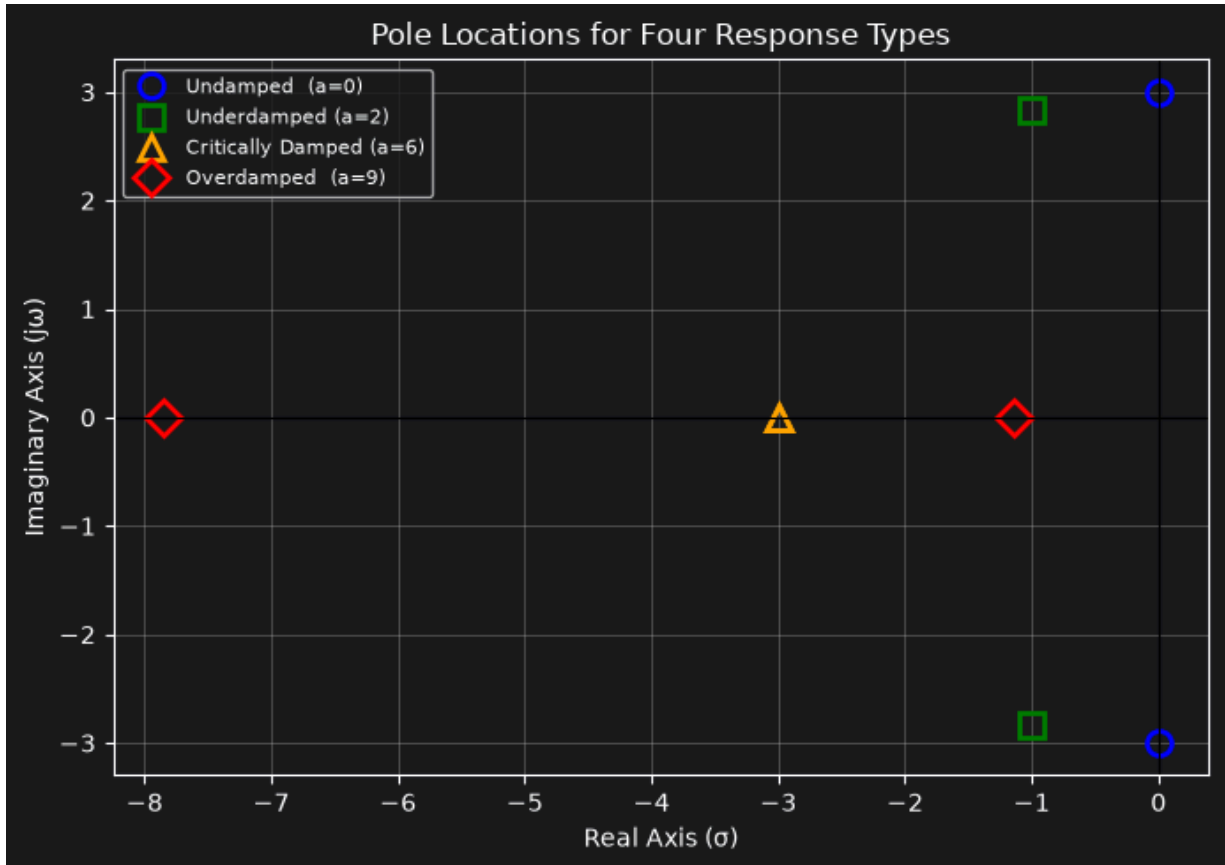
        ax.axhline(0, color='black', linewidth=0.8)

```

```

ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('Real Axis ( $\sigma$ )')
ax.set_ylabel('Imaginary Axis ( $j\omega$ )')
ax.set_title('Pole Locations for Four Response Types')
ax.legend(fontsize=8)
ax.grid(True)
plt.tight_layout()
plt.show()

```



The Canonical (Standard) Form

To quantitatively describe second-order systems, we rewrite the transfer function in **canonical form**:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

where:

- ω_n = **natural frequency** (rad/s) — the oscillation frequency when there is no damping
- ζ (zeta) = **damping ratio** — a dimensionless measure of how quickly oscillations decay

Comparing with $G(s) = \frac{b}{s^2 + as + b}$, we can identify:

$$\omega_n = \sqrt{b} \quad \zeta = \frac{a}{2\omega_n} = \frac{a}{2\sqrt{b}}$$

The damping ratio ζ determines the response type:

ζ	Response Type
$\zeta = 0$	Undamped
$0 < \zeta < 1$	Underdamped
$\zeta = 1$	Critically Damped
$\zeta > 1$	Overdamped

The system poles (from the characteristic equation) are:

$$s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$$

For the underdamped case ($\zeta < 1$), the poles are complex:

$$s_{1,2} = -\zeta\omega_n \pm j\omega_d \quad \text{where } \omega_d = \omega_n\sqrt{1 - \zeta^2}$$

ω_d is the **damped natural frequency**.

```
In [4]: # Example: G(s) = 25 / (s^2 + 6s + 25)
G_ex = control.tf([25], [1, 6, 25])
print('Transfer Function G(s):')
print(G_ex)

# Extract parameters from standard form s^2 + as + b
b_coef = 25
a_coef = 6

wn = np.sqrt(b_coef)          # natural frequency
zeta = a_coef / (2 * wn)      # damping ratio
wd = wn * np.sqrt(1 - zeta**2) # damped natural frequency

print(f'\nNatural Frequency    wn = {wn:.4f} rad/s')
print(f'Damping Ratio        ζ = {zeta:.4f}')
print(f'Damped Nat. Freq.    wd = {wd:.4f} rad/s')

poles = control.pole(G_ex)
print(f'Poles: {np.round(poles, 4)}')

if zeta == 0:
    print('Response type: Undamped')
elif 0 < zeta < 1:
    print('Response type: Underdamped')
elif zeta == 1:
    print('Response type: Critically Damped')
else:
    print('Response type: Overdamped')
```

```
Transfer Function G(s):
<TransferFunction>: sys[20]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
```

25

$s^2 + 6 s + 25$

```
Natural Frequency   wn = 5.0000 rad/s
Damping Ratio       ζ  = 0.6000
Damped Nat. Freq.   wd = 4.0000 rad/s
Poles: [-3.+4.j -3.-4.j]
Response type: Underdamped
```

Underdamped Response Specifications

For an **underdamped** second-order system, the following four specifications quantify transient performance:

Specification	Symbol	Formula
Peak Time	T_p	$T_p = \frac{\pi}{\omega_d}$
Percent Overshoot	$\%OS$	$\%OS = e^{\frac{-\zeta\pi}{\sqrt{1-\zeta^2}}} \times 100\%$
Settling Time	T_s	$T_s = \frac{4}{\zeta\omega_n}$
Rise Time	T_r	$T_r \approx \frac{1.8}{\omega_n}$

We compute and annotate these on the step response of $G(s) = \frac{25}{s^2 + 6s + 25}$.

```
In [5]: # Compute specifications
Tp = np.pi / wd # Peak time
OS = np.exp(-zeta * np.pi / np.sqrt(1 - zeta**2)) * 100 # % Overshoot
Ts = 4 / (zeta * wn) # Settling time
Tr = 1.8 / wn # Rise time (approx)

print(f'Peak Time      Tp = {Tp:.4f} s')
print(f'Percent Overshoot %OS = {OS:.2f}%')
print(f'Settling Time   Ts = {Ts:.4f} s')
print(f'Rise Time (approx) Tr = {Tr:.4f} s')

# Step response
y, t = control.step(G_ex)
```

```

c_final = y[-1]
c_peak  = c_final * (1 + OS / 100)

plt.figure(figsize=(9, 5))
plt.plot(t, y, 'b-', linewidth=2, label='c(t)')

# Final value
plt.axhline(c_final, color='gray', linestyle=':', linewidth=1,
            label=f'Final value = {c_final:.2f}')

# Peak time
plt.axvline(Tp, color='red', linestyle='--',
            label=f'Tp = {Tp:.3f} s')
plt.axhline(c_peak, color='red', linestyle=':', alpha=0.4)

# Rise time
plt.axvline(Tr, color='orange', linestyle='--',
            label=f'Tr ≈ {Tr:.3f} s')

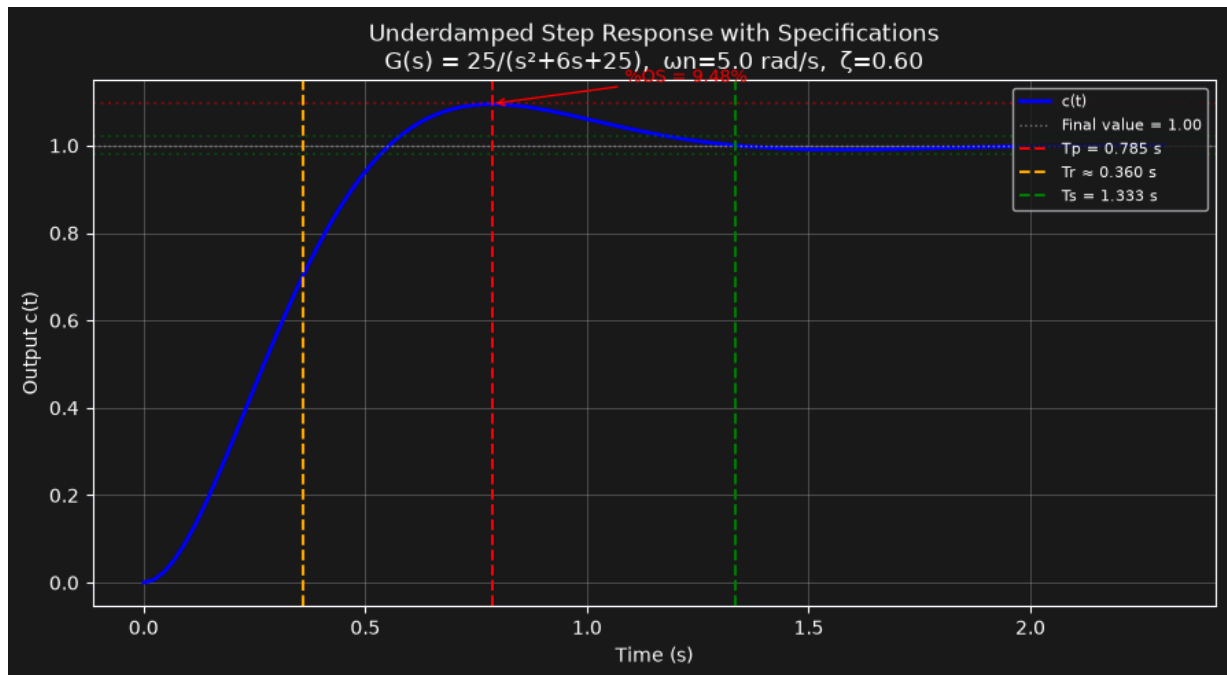
# Settling time with 2% band
plt.axvline(Ts, color='green', linestyle='--',
            label=f'Ts = {Ts:.3f} s')
plt.axhline(0.98 * c_final, color='green', linestyle=':', alpha=0.4)
plt.axhline(1.02 * c_final, color='green', linestyle=':', alpha=0.4)

# Overshoot annotation
plt.annotate(f'%OS = {OS:.2f}%',
            xy=(Tp, c_peak), xytext=(Tp + 0.3, c_peak + 0.05),
            arrowprops=dict(arrowstyle='->', color='red'),
            fontsize=9, color='red')

plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title('Underdamped Step Response with Specifications\n'
          f' $G(s) = 25/(s^2+6s+25)$ ,  $\omega_n={\omega_n:.1f}$  rad/s,  $\zeta={\zeta:.2f}$ ')
plt.legend(loc='upper right', fontsize=8)
plt.grid(True)
plt.tight_layout()
plt.show()

```

Peak Time T_p = 0.7854 s
 Percent Overshoot %OS = 9.48%
 Settling Time T_s = 1.3333 s
 Rise Time (approx) T_r = 0.3600 s



Effect of Damping Ratio on Underdamped Response

For a fixed natural frequency $\omega_n = 5$ rad/s, we compare the step response for several values of ζ in the underdamped range ($0 < \zeta < 1$). A smaller ζ produces more oscillation and greater overshoot.

```
In [6]: wn_fixed = 5
zeta_values = [0.1, 0.3, 0.5, 0.7, 0.9]
colors_zeta = ['navy', 'blue', 'green', 'orange', 'red']

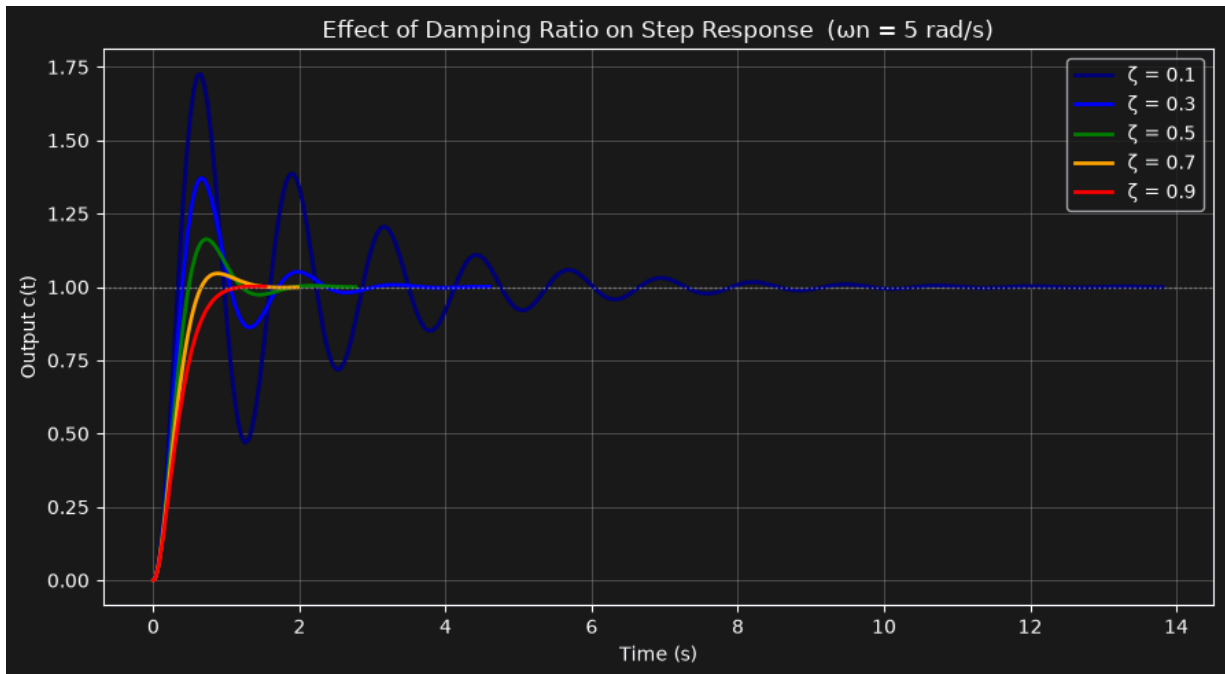
plt.figure(figsize=(9, 5))

for z, color in zip(zeta_values, colors_zeta):
    G = control.tf([wn_fixed**2],
                   [1, 2*z*wn_fixed, wn_fixed**2])
    y, t = control.step(G)
    plt.plot(t, y, color=color, linewidth=2, label=f' $\zeta = \{z\}$ ')

plt.axhline(1, color='gray', linestyle=':', linewidth=1)
plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title(f'Effect of Damping Ratio on Step Response (wn = {wn_fixed} rad/s)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f'\n{"ζ":>6} {"ωd (rad/s)":>12} {"Tp (s)":>10} {"%OS":>8} {"Ts (s)":>10}')
print('-' * 55)
for z in zeta_values:
    wd_z = wn_fixed * np.sqrt(1 - z**2)
    Tp_z = np.pi / wd_z
    OS_z = np.exp(-z * np.pi / np.sqrt(1 - z**2)) * 100
```

```
Ts_z = 4 / (z * wn_fixed)
print(f'{z:>6.1f} {wd_z:>12.4f} {Tp_z:>10.4f} {OS_z:>8.2f} {Ts_z:>10.4f}')
```



ζ	ω_d (rad/s)	T_p (s)	%OS	T_s (s)
0.1	4.9749	0.6315	72.92	8.0000
0.3	4.7697	0.6587	37.23	2.6667
0.5	4.3301	0.7255	16.30	1.6000
0.7	3.5707	0.8798	4.60	1.1429
0.9	2.1794	1.4415	0.15	0.8889

Exercises

Apply what you have learned from the demonstration above to solve the following problems. Add your code in the blank cells provided. Show all computed values and plots.

Exercise 1

For each of the following transfer functions, determine — **by inspection of the poles** — the form of the step response (undamped, underdamped, critically damped, or overdamped). Then verify by plotting all three step responses on a **single figure**.

$$G_A(s) = \frac{16}{s^2 + 16} \quad G_B(s) = \frac{16}{s^2 + 8s + 16} \quad G_C(s) = \frac{16}{s^2 + 3s + 16}$$

- For each system, print the **poles** and state the **response type**.
- Plot all three step responses on the same figure with a legend.
- Do the plots confirm your initial inspection? Explain briefly.

```
In [7]: # Exercise 1 – your code here
print("=== Exercise 1 ===")
```

```

GA = control.tf([16], [1, 0, 16])
GB = control.tf([16], [1, 8, 16])
GC = control.tf([16], [1, 3, 16])

systems_1 = {
    'GA: 16/(s^2+16)': GA,
    'GB: 16/(s^2+8s+16)': GB,
    'GC: 16/(s^2+3s+16)': GC,
}
colors_1 = ['blue', 'red', 'green']

def classify(poles):
    if all(np.isclose(p.real, 0) for p in poles):
        return 'Undamped'
    elif all(np.isreal(p) for p in poles) and len(set(np.round(poles.real, 6))) == 1:
        return 'Critically Damped'
    elif all(np.isreal(p) for p in poles):
        return 'Overdamped'
    else:
        return 'Underdamped'

print(f"\n(a) Pole Analysis:")
for label, G in systems_1.items():
    poles = control.pole(G)
    rtype = classify(poles)
    print(f"    {label}")
    print(f"    Poles: {np.round(poles, 4)} → {rtype}")

# (b) Plot
plt.figure(figsize=(9, 5))
for (label, G), color in zip(systems_1.items(), colors_1):
    y, t = control.step(G)
    plt.plot(t, y, color=color, linewidth=2, label=label)
plt.axhline(1, color='gray', linestyle=':', linewidth=1)
plt.xlabel('Time (s)'); plt.ylabel('Output c(t)')
plt.title('Exercise 1 – Step Responses of GA, GB, GC')
plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()

# (c) Explanation – printed as comment
print("\n(c) Yes, the plots confirm the inspection:")
print("    GA: purely imaginary poles → sustained oscillation (undamped)")
print("    GB: repeated real poles at s=-4 → smooth, no overshoot (critically damped)")
print("    GC: complex conjugate poles → decaying oscillation (underdamped)")

```

=== Exercise 1 ===

(a) Pole Analysis:

GA: $16/(s^2+16)$

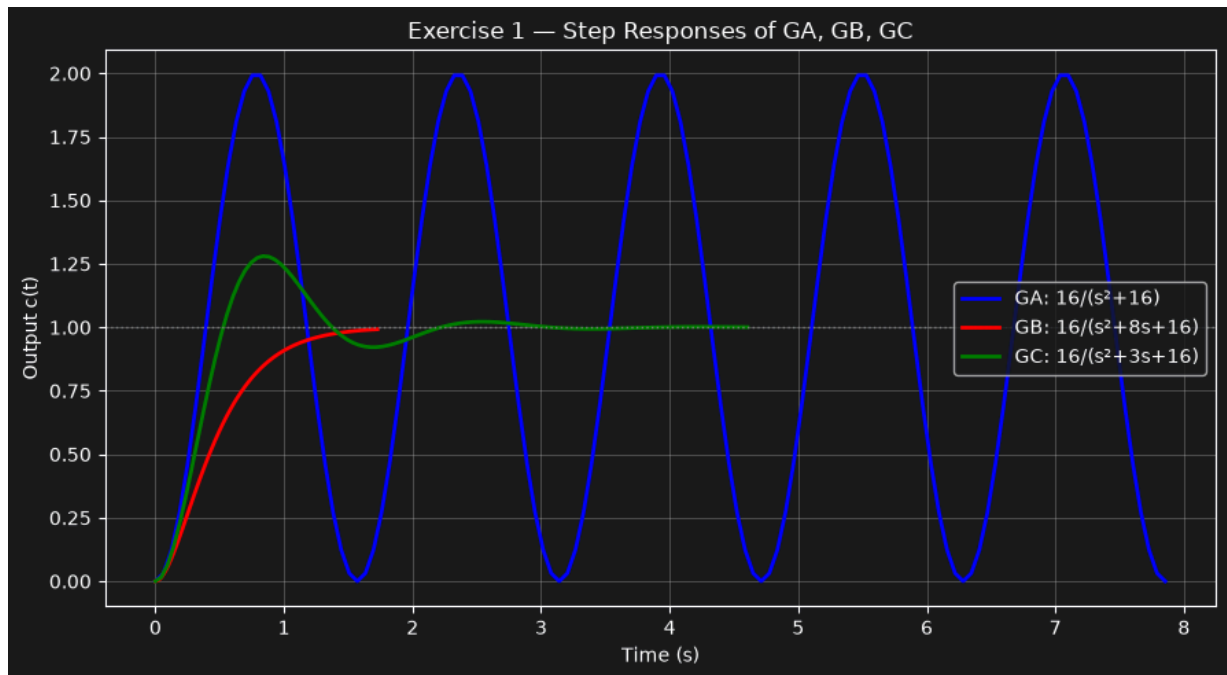
Poles: $[-0.+4.j \ 0.-4.j] \rightarrow$ Undamped

GB: $16/(s^2+8s+16)$

Poles: $[-4.+0.j \ -4.+0.j] \rightarrow$ Critically Damped

GC: $16/(s^2+3s+16)$

Poles: $[-1.5+3.7081j \ -1.5-3.7081j] \rightarrow$ Underdamped



(c) Yes, the plots confirm the inspection:

GA: purely imaginary poles $\rightarrow$ sustained oscillation (undamped)

GB: repeated real poles at $s=-4 \rightarrow$ smooth, no overshoot (critically damped)

GC: complex conjugate poles $\rightarrow$ decaying oscillation (underdamped)

Write your explanation for part (c) here:

Exercise 2

Given the transfer function:

$$G(s) = \frac{36}{s^2 + 4.2s + 36}$$

(a) Identify the **natural frequency** ω_n and **damping ratio** ζ .

(b) Compute the **damped natural frequency** ω_d .

(c) State the **response type** based on ζ .

(d) Compute the **peak time** T_p , **percent overshoot** $\%OS$, **settling time** T_s , and **rise time** T_r .

(e) Plot the step response and **annotate** T_p , T_s , and the 2% settling band.

```
In [8]: # Exercise 2 – your code here
b2, a2 = 36, 4.2
wn2 = np.sqrt(b2)
zeta2 = a2 / (2 * wn2)
wd2 = wn2 * np.sqrt(1 - zeta2**2)

Tp2 = np.pi / wd2
OS2 = np.exp(-zeta2 * np.pi / np.sqrt(1 - zeta2**2)) * 100
Ts2 = 4 / (zeta2 * wn2)
Tr2 = 1.8 / wn2

print(f"(a) wn = {wn2:.4f} rad/s | ζ = {zeta2:.4f}")
print(f"(b) wd = {wd2:.4f} rad/s")
```

```

print(f"(c) Response type: Underdamped ( $0 < \zeta < 1$ )")
print(f"(d)  $T_p = \{Tp2:.4f\}$  s |  $\%OS = \{OS2:.2f\}\%$  |  $T_s = \{Ts2:.4f\}$  s |  $T_r = \{Tr2:.4f\}$  s")

G2 = control.tf([b2], [1, a2, b2])
y2, t2 = control.step(G2)
c_final2 = y2[-1]
c_peak2 = c_final2 * (1 + OS2 / 100)

# (e) Annotated plot
plt.figure(figsize=(9, 5))
plt.plot(t2, y2, 'b-', linewidth=2, label='c(t)')
plt.axhline(c_final2, color='gray', linestyle=':', label=f'Final = {c_final2:.2f}')
plt.axvline(Tp2, color='red', linestyle='--', label=f' $T_p = \{Tp2:.3f\}$  s')
plt.axhline(c_peak2, color='red', linestyle=':', alpha=0.4)
plt.axvline(Ts2, color='green', linestyle='--', label=f' $T_s = \{Ts2:.3f\}$  s')
plt.axhline(0.98 * c_final2, color='green', linestyle=':', alpha=0.4)
plt.axhline(1.02 * c_final2, color='green', linestyle=':', alpha=0.4)
plt.annotate(f'%OS = {OS2:.2f}%',
             xy=(Tp2, c_peak2), xytext=(Tp2 + 0.15, c_peak2 + 0.04),
             arrowprops=dict(arrowstyle='->', color='red'), fontsize=9, color='red')
plt.xlabel('Time (s)'); plt.ylabel('Output c(t)')
plt.title(f'Exercise 2 — Step Response  $\omega_n=\{wn2:.1f\}$  rad/s,  $\zeta=\{zeta2:.3f\}$ ')
plt.legend(fontsize=8); plt.grid(True); plt.tight_layout(); plt.show()

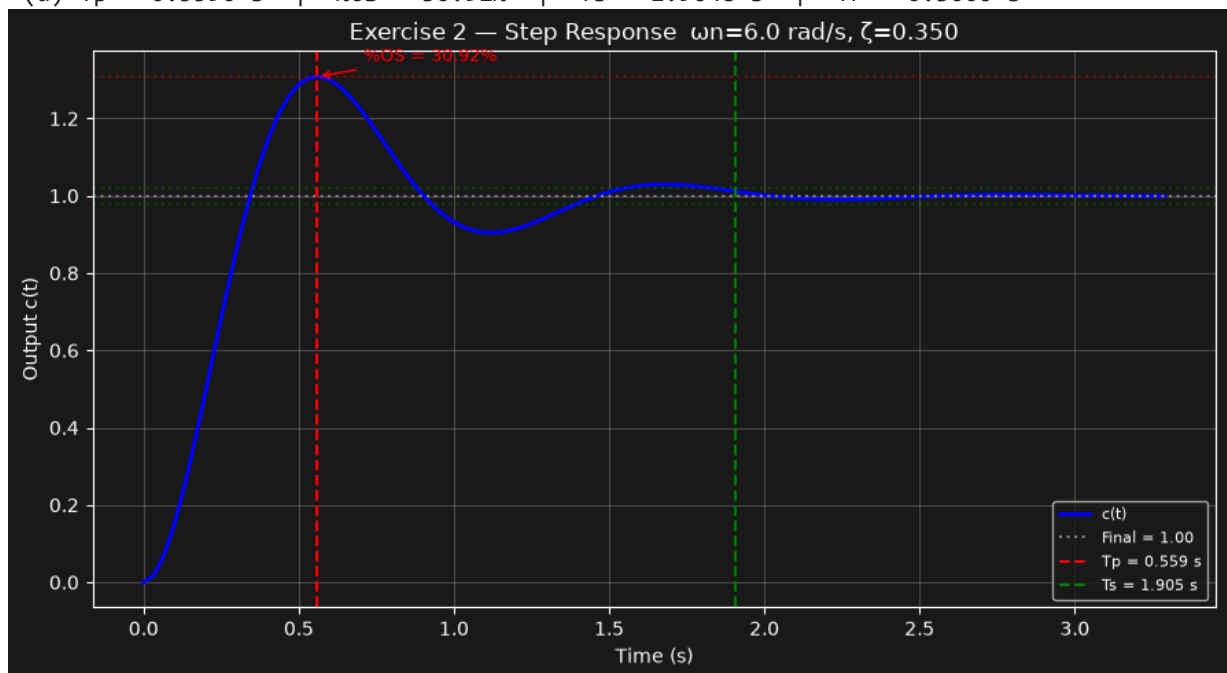
```

(a) $\omega_n = 6.0000$ rad/s | $\zeta = 0.3500$

(b) $\omega_d = 5.6205$ rad/s

(c) Response type: Underdamped ($0 < \zeta < 1$)

(d) $T_p = 0.5590$ s | $\%OS = 30.92\%$ | $T_s = 1.9048$ s | $T_r = 0.3000$ s



Exercise 3

A second-order system is required to meet the following specifications:

- **Natural frequency:** $\omega_n = 8$ rad/s
- **Damping ratio:** $\zeta = 0.4$

- (a) Write the **transfer function** $G(s)$ in canonical form.
- (b) Compute T_p , %OS, T_s , and T_r .
- (c) Plot the **step response** and annotate T_p , T_s , and the percent overshoot.
- (d) If the damping ratio is increased to $\zeta = 0.7$ (keeping $\omega_n = 8$), how do the specifications change? Plot both responses on the same figure and compare.

```
In [9]: # Exercise 3 – your code here
print("\n=== Exercise 3 ===")

wn3 = 8

def specs(wn, zeta):
    wd = wn * np.sqrt(1 - zeta**2)
    Tp = np.pi / wd
    OS = np.exp(-zeta * np.pi / np.sqrt(1 - zeta**2)) * 100
    Ts = 4 / (zeta * wn)
    Tr = 1.8 / wn
    return wd, Tp, OS, Ts, Tr

for z_val in [0.4, 0.7]:
    wd3, Tp3, OS3, Ts3, Tr3 = specs(wn3, z_val)
    G = control.tf([wn3**2], [1, 2*z_val*wn3, wn3**2])
    print(f"\n  ζ = {z_val}:")
    print(f"    (a) G(s) = {wn3**2}/(s^2 + {2*z_val*wn3}s + {wn3**2})")
    print(f"    (b/d) Tp={Tp3:.4f}s  %OS={OS3:.2f}%  Ts={Ts3:.4f}s  Tr={Tr3:.4f}s")

# (c)+(d) Both on same figure
plt.figure(figsize=(9, 5))
zeta_pairs = [(0.4, 'blue', 'ζ=0.4'), (0.7, 'orange', 'ζ=0.7')]

for z_val, color, lbl in zeta_pairs:
    G = control.tf([wn3**2], [1, 2*z_val*wn3, wn3**2])
    y, t = control.step(G)
    c_f = y[-1]
    wd3, Tp3, OS3, Ts3, Tr3 = specs(wn3, z_val)
    c_pk = c_f * (1 + OS3 / 100)
    plt.plot(t, y, color=color, linewidth=2, label=lbl)
    plt.axvline(Tp3, color=color, linestyle='--', alpha=0.6, label=f'Tp ({lbl}) = {Tp3:.4f}s')
    plt.axvline(Ts3, color=color, linestyle=':', alpha=0.6, label=f'Ts ({lbl}) = {Ts3:.4f}s')

plt.axhline(1, color='gray', linestyle=':', linewidth=1)
plt.xlabel('Time (s)'); plt.ylabel('Output c(t)')
plt.title(f'Exercise 3 – ζ=0.4 vs ζ=0.7 (wn={wn3} rad/s)')
plt.legend(fontsize=7); plt.grid(True); plt.tight_layout(); plt.show()

print("\n (d) Comparison:")
print("    ζ=0.7 settles faster and has far less overshoot than ζ=0.4.")
print("    Higher damping suppresses oscillations at the cost of a slightly")
print("    slower rise – but the overall transient is cleaner and more stable.")
```

=== Exercise 3 ===

$\zeta = 0.4$:

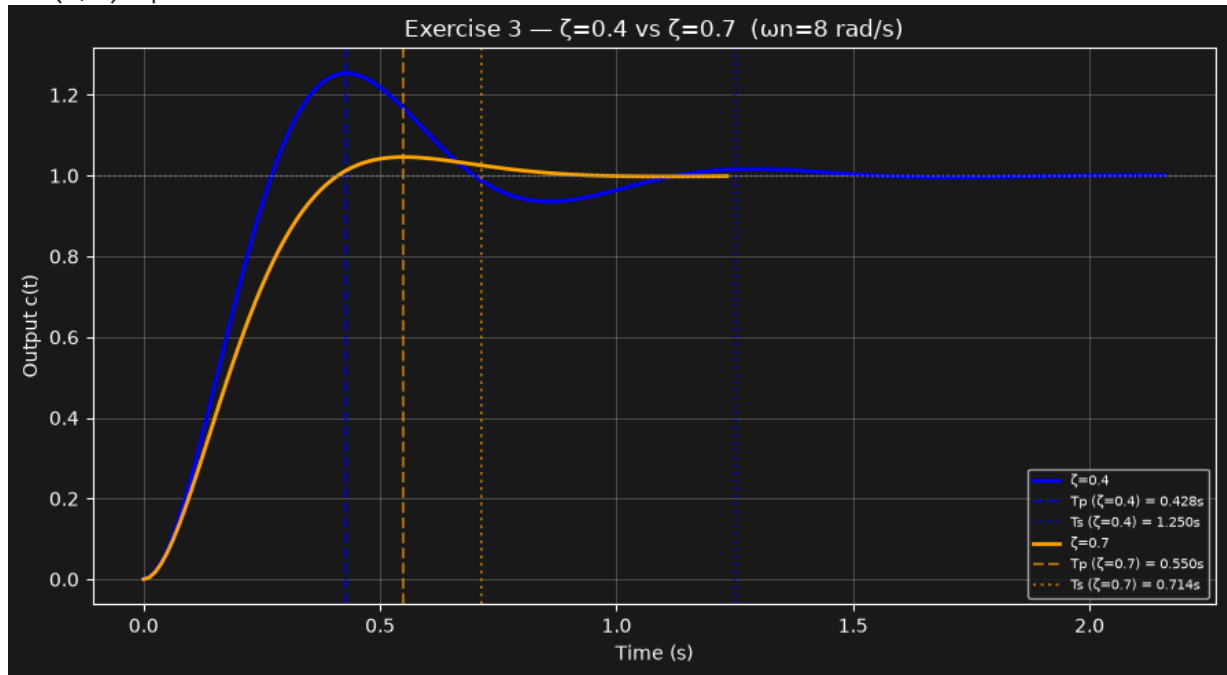
(a) $G(s) = 64/(s^2 + 6.4s + 64)$

(b/d) $T_p=0.4285s$ $\%OS=25.38\%$ $T_s=1.2500s$ $T_r=0.2250s$

$\zeta = 0.7$:

(a) $G(s) = 64/(s^2 + 11.2s + 64)$

(b/d) $T_p=0.5499s$ $\%OS=4.60\%$ $T_s=0.7143s$ $T_r=0.2250s$



(d) Comparison:

$\zeta=0.7$ settles faster and has far less overshoot than $\zeta=0.4$.

Higher damping suppresses oscillations at the cost of a slightly slower rise – but the overall transient is cleaner and more stable.

Write your comparison for part (d) here:

Exercise 4

For each of the following transfer functions, find the **natural frequency**, **damping ratio**, and **response type**. Then use these values to compute T_p , $\%OS$, T_s , and T_r **only for the underdamped cases**. Present your results in a printed summary table.

$$G_1(s) = \frac{12}{s^2 + 8s + 12} \quad G_2(s) = \frac{16}{s^2 + 2s + 16} \quad G_3(s) = \frac{4}{s^2 + 4s + 4}$$

```
In [10]: # Exercise 4 – your code here
print("\n=== Exercise 4 ===")

systems_4 = {
    'G1: 12/(s^2+8s+12)': ([12], [1, 8, 12]),
    'G2: 16/(s^2+2s+16)': ([16], [1, 2, 16]),
    'G3: 4/(s^2+4s+4)': ([4], [1, 4, 4]),
}
```

```

print(f"\n{'System':<22} {'wn':>6} {'ζ':>7} {'Type':<20} {'Tp':>8} {'%OS':>8} {'Ts':>8}")
print('-' * 92)

for label, (num, den) in systems_4.items():
    b_c = den[2]
    a_c = den[1]
    wn_c = np.sqrt(b_c)
    zeta_c = a_c / (2 * wn_c)
    G = control.tf(num, den)
    poles_c = control.pole(G)
    rtype_c = classify(poles_c)

    if 0 < zeta_c < 1: # underdamped only
        wd_c, Tp_c, OS_c, Ts_c, Tr_c = specs(wn_c, zeta_c)
        print(f"{'label':<22} {'wn_c':>6.3f} {'zeta_c':>7.4f} {'rtype_c':<20} "
              f"{'Tp_c':>8.4f} {'OS_c':>7.2f}% {'Ts_c':>8.4f} {'Tr_c':>8.4f}")
    else:
        print(f"{'label':<22} {'wn_c':>6.3f} {'zeta_c':>7.4f} {'rtype_c':<20} "
              f"{'-':>8} {'-':>8} {'-':>8} {'-':>8}")

```

=== Exercise 4 ===

System s	Tr	wn	ζ	Type	Tp	%OS	T
G1: 12/(s ² +8s+12)	—	3.464	1.1547	Overdamped	—	—	—
G2: 16/(s ² +2s+16)	0.4500	4.000	0.2500	Underdamped	0.8112	44.43%	4.000
G3: 4/(s ² +4s+4)	—	2.000	1.0000	Critically Damped	—	—	—